

EARTH: An Efficient MoE Accelerator with Entropy-Aware Speculative Prefetch and Pattern Reuse

Fangxin Liu*

Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China
liufangxin@sjtu.edu.cn

Ning Yang*

Shanghai Jiao Tong University
Shanghai, China
yn937391832@sjtu.edu.cn

Jingkui Yang

National University of Defense
Technology
Shanghai Qi Zhi Institute
China
yangjingkui2001@nudt.edu.cn

Zongwu Wang

Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China
wangzongwu@sjtu.edu.cn

Chenyang Guan

Shanghai Jiao Tong University
Shanghai, China
guanchenyang@sjtu.edu.cn

Yu Feng

Shanghai Jiao Tong University
Shanghai, China
y-feng@sjtu.edu.cn

Li Jiang

Shanghai Jiao Tong University
Shanghai Qi Zhi Institute
Shanghai, China
ljiang_cs@sjtu.edu.cn

Haibing Guan[†]

Shanghai Jiao Tong University
Shanghai, China
hbguan@sjtu.edu.cn

Abstract

Mixture-of-Experts (MoE) models significantly reduce computation in large language models by activating only a subset of experts per input token, but they introduce severe memory bottlenecks due to the large number of expert parameters. Existing offloading and prefetching strategies either incur accuracy loss, prohibitively high memory traffic, or high decoding overhead, limiting deployment on resource-constrained hardware. In this work, we present EARTH, a hardware–software co-design that addresses these challenges through three key innovations. First, we propose a dual-entropy encoding scheme that decomposes each expert into a high-information base and a delta component, enabling compact storage while preserving accuracy via adaptive precision management. Second, we introduce a delta-aware speculative prefetching and reuse mechanism that preloads base components of predicted experts and selectively fetches deltas, reusing previously computed delta patterns to reduce memory traffic and redundant computation. Third, we design a hardware accelerator that is co-designed

to efficiently support this encoding and prefetching strategy, optimizing execution order, parallelism, and memory utilization. Across representative MoE workloads, EARTH reduces data movement overhead, improves prefetch efficiency, and achieves up to 2.10× speedup compared to state-of-the-art baselines, while maintaining high model accuracy.

CCS Concepts: • Computer systems organization → Neural networks; • Hardware → Hardware accelerators; • Computing methodologies → Artificial intelligence.

Keywords: Mixture-of-Experts (MoE), accelerator, hardware–software co-design, speculative prefetching

ACM Reference Format:

Fangxin Liu, Ning Yang, Jingkui Yang, Zongwu Wang, Chenyang Guan, Yu Feng, Li Jiang, and Haibing Guan. 2026. EARTH: An Efficient MoE Accelerator with Entropy-Aware Speculative Prefetch and Pattern Reuse. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3779212.3790155>

1 Introduction

Large Language Models (LLMs), such as GPT-4 [1], LLaMA [8], and DeepSeek-R1 [13], have demonstrated remarkable performance in tasks like machine translation, summarization, reasoning, and code generation. Scaling model size generally improves accuracy, but incurs substantial computational and memory costs. For example, serving a 175B-parameter model for real-time inference requires over 350 GB of GPU memory [37], exceeding the capacity of a single accelerator

*Both authors contributed equally to the paper.

[†]Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790155>

and posing deployment challenges in resource-constrained environments.

The Mixture-of-Experts (MoE) architecture offers a compelling alternative to dense LLMs by selectively activating a subset of experts per input token, reducing computational overhead while maintaining or surpassing the performance of dense models [19, 28]. For instance, models like Switch-Transformer [10] achieve state-of-the-art results with fewer activated parameters than their dense counterparts. However, MoE models introduce significant memory challenges, particularly for edge deployment. In MoE-based Transformers, multi-layer perceptron (MLP) layers are replaced with MoE layers, each comprising a top-k gating network and multiple expert subnetworks. During inference, the gating network selects the top-k experts for computation, lowering the computational cost. Yet, the large number of experts—often up to 75× more parameters than a FLOPs-equivalent dense model, as seen in Google’s Switch-Transformer compared to T5 [29]—significantly increases memory requirements. This makes it infeasible to store all expert parameters in GPU memory, creating a critical bottleneck for memory-efficient deployment.

Figure 1 illustrates this effect using the Qwen3-30B-A3B model [43] with INT8 quantization on a DDR5-6400 system. Figure 1(a) depicts the MoE inference pipeline, including gating, expert fetching, computation, and aggregation. Figure 1(b) quantifies execution time: expert fetching consumes ~88% of total cycles, while gating, computation, and aggregation together account for less than 12%. This shows that data movement, rather than computation, is the dominant bottleneck, motivating architectures and scheduling strategies specifically designed to mitigate expert-fetch latency.

Existing solutions address memory limitations through expert parallelism, compression, or offloading. Expert parallelism distributes experts across multiple accelerators, but increases hardware cost and complexity. Compression techniques, such as pruning or quantization [12, 17, 40], reduce memory footprints but may degrade accuracy or require expensive retraining. Offloading methods [36, 42] transfer infrequently used expert weights to slower memory tiers (e.g., CPU or NVMe), providing hardware-friendly alternatives. However, prior offloading relies on static or heuristic prefetching, which cannot fully exploit spatial and temporal locality in expert access patterns, resulting in low prefetch accuracy, inefficient bandwidth use, and pipeline stalls. Specifically, simplistic prefetching schemes cannot adapt to dynamic runtime behavior, while complex strategies introduce overheads that hinder effective compute-data overlap.

Our study addresses the following questions to make MoE offloading efficient and practical:

- **Can expert parameters be dynamically represented to reduce memory footprint without losing inference quality?** Yes. We identify sensitive experts

highly correlated with inference quality and apply high-fidelity representation for these, while using low-fidelity representation for others, improving performance under tight memory budgets (Sec. 3.2).

- **Can expert representation be maintained without complex encoding/decoding overhead?** We propose a dual-entropy encoding scheme, decomposing each expert into a high-information *base* and a *delta* component, enabling compact storage and efficient reconstruction (Sec. 4.2).
- **How to conduct efficient prefetching under limited buffer capacity?** We introduce a delta-aware speculative prefetching and reuse strategy. The base part of predicted experts is loaded proactively, while deltas for sensitive experts are fetched just-in-time. For less important experts, previously computed delta patterns are reused, reducing both memory traffic and computation (Sec. 5.2).

Unlike conventional offloading schemes, EARTH dynamically loads experts based on their sensitivity, ensuring that high-fidelity representation is applied to the most critical experts while reducing storage and computation for less important ones. The key insight is that by separating experts into a high-information base and a delta component, EARTH avoids complex multi-precision storage and decoding overhead, enabling more flexibility for fine-grained scheduling. Furthermore, because only the base components of predicted experts are loaded initially, more experts can reside in the limited on-chip buffer, increasing prefetch success rates. Finally, EARTH is hardware-friendly and can be implemented by modest modifications to widely used accelerator architectures. Notably, EARTH distinguishes itself from existing ML accelerators by targeting MoE’s unique bottlenecks, including irregular expert accesses, dynamic routing, and dominant data movement as Figure 1(b) shows. These key challenges are overlooked by general-purpose LLM accelerators [6, 36] and even prior MoE-specific designs [5, 32, 44]. Unlike general accelerators that optimize computation alone, EARTH’s hardware-software co-design integrates encoding, prefetching, and reuse to minimize data movement. Unlike prior MoE accelerators relying on static sparsity, model modifications, or CPU/GPU scheduling, EARTH adapts dynamically to expert sensitivity and access patterns without retraining or structural changes. In summary, our contributions are as follows:

- We propose a dual-entropy encoding scheme that decomposes each high-precision expert into a high-information *base* and a *delta* component, enabling compact storage.
- We design a delta-aware speculative prefetching mechanism that predicts frequently used experts and preloads their base components, fetching deltas only when needed. This reduces memory traffic while maintaining high accuracy for sensitive experts.

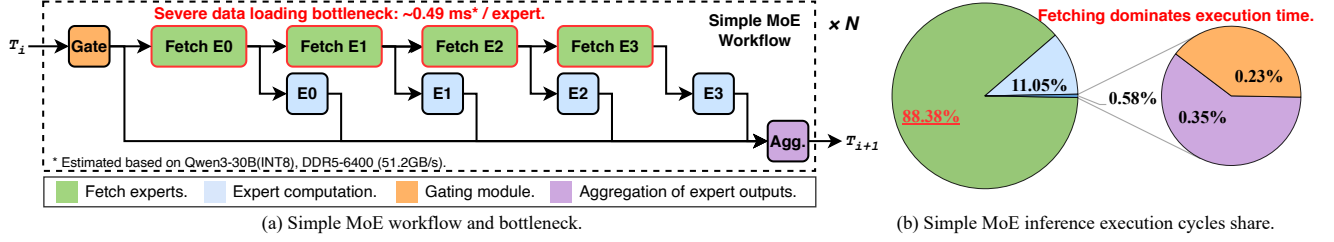


Figure 1. MoE workflow and execution time breakdown. (a) illustrates the inference pipeline with gating, expert fetching, expert computation, and aggregation. (b) shows the execution time distribution from a representative profiling study of MoE inference, where expert fetching accounts for ~88% of the total execution time, highlighting the severe data-loading bottleneck.

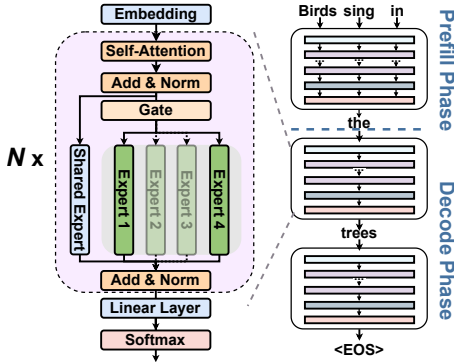


Figure 2. MoE model inference procedure with 1 shared expert and top-2 routed experts activated per token.

- We introduce a pattern-based reuse paradigm that caches partial experts across tokens, eliminating redundant delta fetches during speculative execution.
- We develop the EARTH accelerator, a hardware architecture tailored to support dual-entropy encoding and speculative prefetching. By exploiting encoding sparsity and reusing previously computed results, EARTH efficiently schedules dynamic expert selection and data movement to maximize throughput and minimize memory stalls.

2 Background

2.1 Mixture-of-Experts(MoEs)

The MoEs architecture leverages sparse activation to enhance the efficiency of large-scale neural networks. Unlike traditional dense models, MoE employs multiple sub-models, termed “experts,” each specializing in distinct sub-tasks. This section uses Transformer-based models, such as LLaMA [35], GPT [1], Mixtral [20], DBRX [4], and Grok [38], as examples to illustrate MoE’s structure and operation. A standard Transformer model comprises stacked Transformer layers, each containing a Multi-Head Attention (MHA) block followed by a Multi-Layer Perceptron (MLP) block. In contrast,

an MoE layer replaces the MLP block with an MoE block to introduce sparsity, as depicted in Figure 2.

An MoE block consists of two primary components: a gating operation and multiple experts. The gating operation, typically implemented with one or two feed-forward layers, a softmax function, and a top- k selection mechanism, routes each input token to k selected experts. Each expert, structurally akin to an MLP, is a feed-forward neural network tailored to specific sub-tasks. During inference, input tokens pass through the gating operation, which assigns them to the appropriate experts based on learned routing decisions.

Recent MoE designs incorporate heterogeneous experts to optimize computational efficiency [34, 38], as shown in Figure 2. These experts are categorized into two types: (1) **routed experts**, dynamically selected by the gating operation with outputs aggregated using soft weights, and (2) **shared experts**, activated for every forward pass. This hybrid approach balances computational cost and model performance, enabling efficient MoE deployment on resource-constrained hardware. Since shared experts are always activated in the calculation, they are preloaded by default and excluded from our analysis.

2.2 Offloading Optimization in MoE

Offloading techniques for MoE models aim to mitigate the computational and memory demands of large-scale expert models during edge inference. Several approaches leverage caching and prefetching to optimize performance on resource-constrained hardware. Mixtral-Offloading [9] dynamically caches expert parameters by offloading infrequently used experts to host memory and uploading activated experts to GPU devices, using mixed-precision quantization to accelerate inference. MoE-Infinity [39] employs activation-aware prefetching to reduce I/O latency by predicting sequence-level expert activation patterns. EdgeMoE [41] reduces memory footprint through expert-specific bit-width adaptation, minimizing accuracy degradation, and preloads anticipated experts via a compute-I/O pipeline. SiDA-MoE [7] predicts expert activation using a hash-based approach for efficient

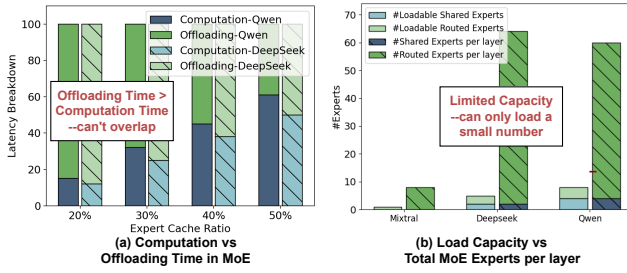


Figure 3. Offloading bottlenecks in MoE inference under constrained memory. (a) Offloading latency exceeds computation time, contributing significant overhead. (b) Low expert capacity necessitates frequent reloading, degrading memory efficiency.

runtime preloading. Pre-gated MoE [18] optimizes prefetching with a predictive pre-gating mechanism. DAOP [42] dynamically allocates experts between CPU and GPU based on per-sequence activation patterns, pre-computing predicted experts on CPUs to reduce transfer latency. HybriMoE [45] balances CPU-GPU workloads through dynamic intra-layer scheduling, incorporating inter-layer prefetching and score-based caching to address expert activation variability. Despite these advancements, existing strategies often fail to fully mask expert loading overhead in large-scale MoE models, as limited memory bandwidth and unpredictable activation patterns remain significant bottlenecks in heterogeneous systems.

3 Motivation

We begin by characterizing the performance bottlenecks of MoE models and identifying the key factors contributing to their overheads.

3.1 Performance Characteristics

The sparse activation in MoE models, where only a subset of experts is selected per token, reduces computation but introduces significant system-level challenges. Dynamic expert selection results in irregular memory accesses and frequent transfers between on-chip and off-chip memory, which often dominate inference latency. As shown in Figure 3(a), profiling Qwen1.5-MoE-A2.7B [34] and DeepSeek-V2-Lite [26] under constrained memory budgets shows that offloading overhead can surpass the actual computation time, leaving limited opportunities to overlap transfers with expert execution.

This challenge is amplified by the limited on-chip memory capacity of FPGA accelerators. As illustrated in Figure 3(b), only a small subset of experts can be cached at once, forcing repeated fetches from external DRAM whenever new experts are activated. This not only wastes scarce on-chip capacity

but also creates redundant transfers across the memory interface.

To quantify these effects, we model the expert loading time during MoE inference. Consider an MoE layer with n experts, top- k selection per token, and prefetching of p experts. The loading time is:

$$T_{load} = (k - R)\bar{t} \quad (1)$$

where R is the number of successfully prefetched experts and $\bar{t}$ is the average time to load one expert. Suppose routed experts are uniformly sampled from n , the expected loading time is:

$$\mathbb{E}[T_{load}] = k(1 - \frac{p}{n})\bar{t} \quad (2)$$

This model highlights a fundamental bottleneck: load latency scales with both prefetch effectiveness and single-expert load time, underscoring the importance of fast and adaptive offloading strategies.

3.2 Performance Bottlenecks

Analysis of the expert loading time model (Equation 2) highlights two dominant factors in MoE inference efficiency: (1) the average expert load time ($\bar{t}$), including penalties from prefetch misses, and (2) the number of experts that can be prefetched (p). Our study focuses on these two aspects.

Observation 1: Expert Sensitivity. To investigate the intrinsic properties of MoE parameters, we analyze expert robustness by injecting additive noise. Specifically, we evaluate all experts in Qwen1.5-MoE-A2.7B and measure the cosine distance between outputs before and after perturbation. As shown in Figure 4(a), experts exhibit widely varying levels of sensitivity—even within the same layer. A closer inspection of eight selected experts (Figure 4(b)) confirms that some experts can tolerate significant perturbations, while others are fragile. This heterogeneity suggests an opportunity: robust experts can be quantized more aggressively at runtime without severely impacting accuracy.

The challenge, however, lies in realizing such adaptive representation under tight memory constraints. Maintaining multiple precision versions of each expert (e.g., INT8 and INT4) would double storage and transfer costs, which is infeasible. On the other hand, storing only one copy and statically truncating parameters (e.g., keeping only the high 4 bits) causes substantial information loss. To further examine this, we analyze bit-level redundancy in INT8 representation. As shown in Figure 5, while high-order bits contribute to the numeric range, their information entropy is significantly lower, indicating sparse utilization. Naively discarding or approximating these bits amplifies error, highlighting the need for a more fine-grained, sensitivity-aware encoding scheme.

Observation 2: Prefetch Capacity. Prefetching is another key factor in mitigating offloading overhead. Figure 6(b)

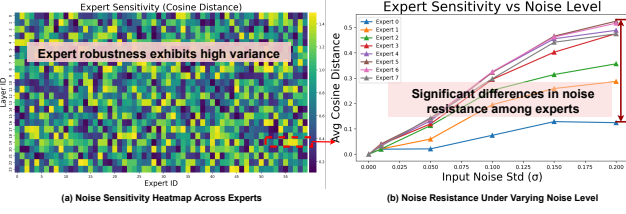


Figure 4. MoE expert robustness profiling under additive noise in Qwen1.5-MoE-A2.7B. (a) Distribution of sensitivity levels across all experts, showing high variance: some experts remain robust while others are highly sensitive. (b) Noise response of eight selected experts. Tolerant experts (e.g., Expert 0) maintain output similarity under increasing noise, whereas sensitive experts (e.g., Expert 6, 7) degrade sharply at low noise.

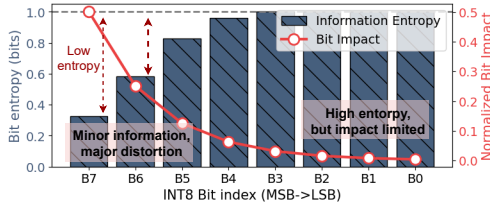


Figure 5. Information entropy and contribution of each bit in an INT8 representation.

shows that increasing the prefetch budget reduces the number of experts that must be loaded per step. For example, in *DeepSeek-V2-Lite-Chat*, enlarging the prefetch window from 6 to 16 lowers the average number of loaded experts from 5.22 to 4.09. Similar improvements are observed in *Qwen-MoE*.

However, prefetch capacity is fundamentally bounded by on-chip memory and bandwidth. To overcome this limitation, we revisit the sliceable parameter format before. Since entropy is concentrated in the higher-order part of the representation, it becomes feasible to speculatively prefetch only the most informative slices of each expert instead of the full parameter set. This approach enables more experts to be prefetched under the same memory budget, while preserving model accuracy.

Together, these observations highlight two intertwined bottlenecks in MoE offloading: (1) heterogeneous expert sensitivity allows for runtime quantization but requires efficient, storage-friendly encoding, and (2) prefetching is effective but capacity-constrained. These findings motivate a dynamic expert encoding and prefetch strategy that jointly exploits robustness and entropy concentration to reduce transfer overhead under hardware constraints.

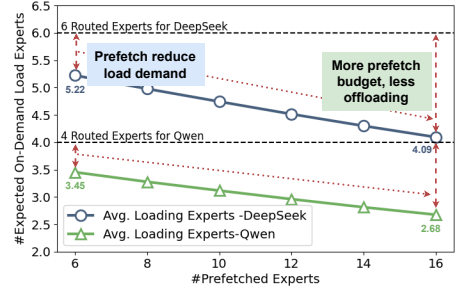


Figure 6. Impact of prefetch budget on expert loading demand in Qwen1.5-MoE-A2.7B and DeepSeek-V2-Lite-Chat.

4 Algorithm

4.1 Overview

The analysis in Section 3 highlights two intertwined bottlenecks: (1) representing experts in a compact form without additional storage, and (2) reducing offloading cost by improving prefetch efficiency. To address these issues, we propose a dynamic expert encoding and prefetch strategy that combines *dual-entropy encoding* with *speculative prefetching*.

The key idea is to exploit the entropy concentration in high-order bits of expert weights. Dual-entropy encoding produces a sliceable format consisting of a **base part** (high-order) and a **delta part** (low-order), which is efficient to store and compute. This enables prefetching strategies that load only the base part speculatively, reducing transfer overhead while maintaining model accuracy. In addition, gating weights guide whether to fetch the delta part, allowing low-weight experts to be approximated using only the base. Finally, we observe that delta values often exhibit reuse patterns, which motivates a lightweight reuse mechanism to reduce bandwidth cost further. Section 4.2 describes dual-entropy encoding, Section 4.3 details speculative prefetching guided by routing patterns and gating weights, and Section 4.4 analyzes delta reuse opportunities.

4.2 Dual-Entropy Encoding

To dynamically represent MoE experts, we introduce *dual-entropy encoding*. For standard INT8 quantized experts, each 8-bit weight w is decomposed into a 4-bit high-order **base** w_b and a 4-bit low-order **delta** w_d , such that $w = (w_b \ll 4) + w_d$. Formally, each weight is expressed as a sum of multiple terms, where a b -bit code $c = (c_0, c_1, \dots, c_{b-1})$ represents:

$$x = \sum_{i=0}^{b-1} s(c_i) \cdot 2^i \quad (3)$$

with $s(c_i) = +1$ if $c_i = 1$ and $s(c_i) = -1$ if $c_i = 0$.

This representation offers two useful properties. First, it preserves homomorphism for addition and multiplication so that partial computations can be directly combined. Second, it scales naturally across bit-widths with consistent structure.

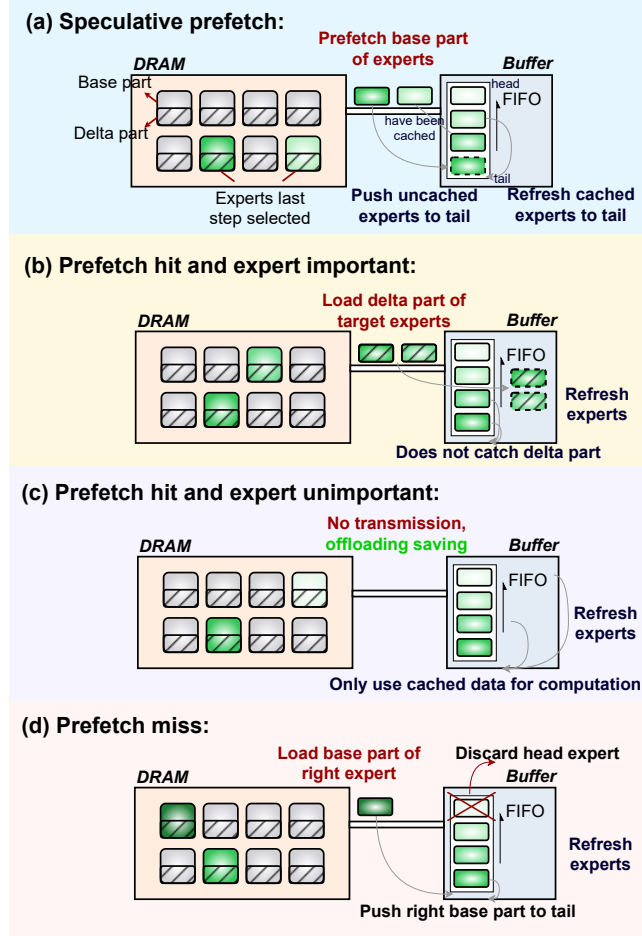


Figure 7. Workflow of EARTH's speculative prefetching. (a) Base parts of candidate experts are prefetched into a FIFO DRAM buffer; cached entries are refreshed while new ones are appended. (b) On a prefetch hit for an important expert, EARTH loads its delta part to enable full-precision computation. (c) On a hit for an unimportant expert, EARTH reuses only the cached base part, saving bandwidth. (d) On a prefetch miss, the correct expert's base part is loaded while the FIFO head is discarded, minimizing penalty.

More importantly, due to the distribution of neural network weights, the high-order bits exhibit higher entropy and capture most of the information. We therefore partition the code into a **base part** (high-order) and a **delta part** (low-order), which form the basis of our dual-entropy representation. Meanwhile, this 4/4 split is hardware-friendly: First, it ensures memory alignment for efficient 1-byte accesses, avoiding unaligned memory transactions that introduce latency. Second, it matches the native data width of common PEs in accelerator architectures, enabling direct integration with existing compute pipelines without additional conversion overhead.

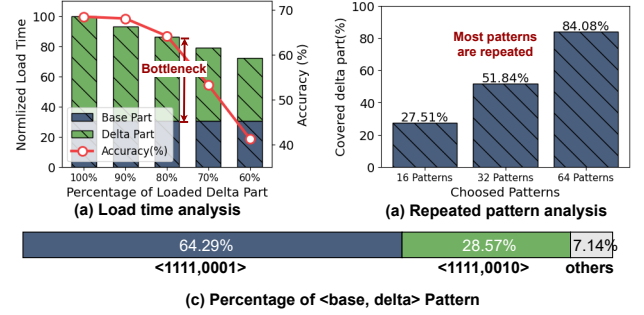


Figure 8. Bottleneck and pattern analysis. (a) Load time analysis. Delta part accounts for over 60% of total load time, becoming the new bottleneck with speculative prefetching. (b) Repeated pattern analysis. A small fraction of base-delta patterns accounts for most MoE parameters. (c) Percentage of <base, delta> pattern for base part <1111>. Dominating patterns show high redundancy for reuse.

4.3 Delta-Aware Speculative Prefetching

Building on the dual-entropy encoding, we design a delta-aware speculative prefetching strategy in EARTH to maximize expert availability under strict memory and bandwidth budgets. The key insight is that high-order *base* bits carry most of the information, while low-order *delta* bits mainly refine accuracy. This enables us to prefetch only the base parts speculatively and decide delta loading adaptively.

Our approach leverages routing history from the gating layer to predict which experts are likely to be selected for the next token. We preload only their base parts into an on-chip buffer, which keeps prefetch overhead low and allows more experts to be cached. Once the actual routing is known, we distinguish three cases (Figure 7):

- **Hit with important expert:** If the prediction is correct and the expert's gating weight exceeds a threshold, we fetch the delta part to complete the representation, ensuring accuracy.
- **Hit with unimportant expert:** If the gating weight is low, we skip the delta fetch and use only the base part. This avoids transmission without noticeably hurting accuracy.
- **Miss:** If prediction fails, we load only the base part of the correct expert. The penalty is minimal, since full deltas are never prefetched speculatively.

The gating weight threshold is calibrated offline, ensuring that skipping deltas does not degrade accuracy beyond 1%. This delta-aware strategy, combined with sliceable encoding, enables more experts to be prefetched within the same budget and significantly reduces transfer overhead.

4.4 Delta Bottleneck after Prefetching

Speculative prefetching significantly reduces base-loading overhead, allowing more experts to be kept on-chip with

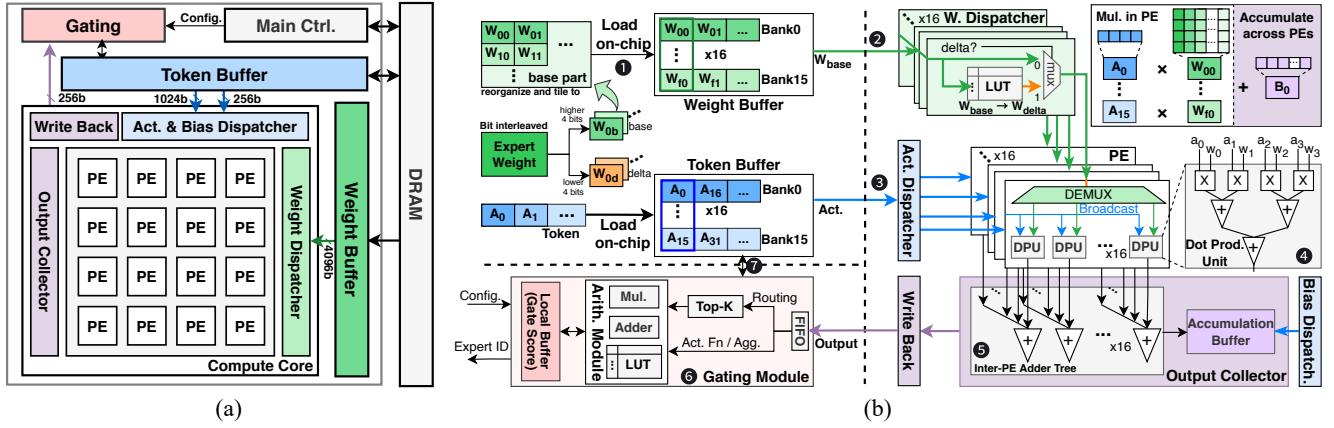


Figure 9. Proposed EARTH accelerator architecture. (a) *Architecture overview*: EARTH integrates a gating module, token and weight buffers, and a compute core with 16 PEs. Dedicated dispatchers manage weights, activations, and bias, while an output collector aggregates results. This organization covers the full MoE inference pipeline, enabling low-latency expert selection, high-throughput weight transfer, and sustained PE utilization. (b) *Dataflow and memory layout*: Execution leverages on-chip tiling, bit-interleaving of expert parameters, and LUT-based reconstruction from base/delta partitions.

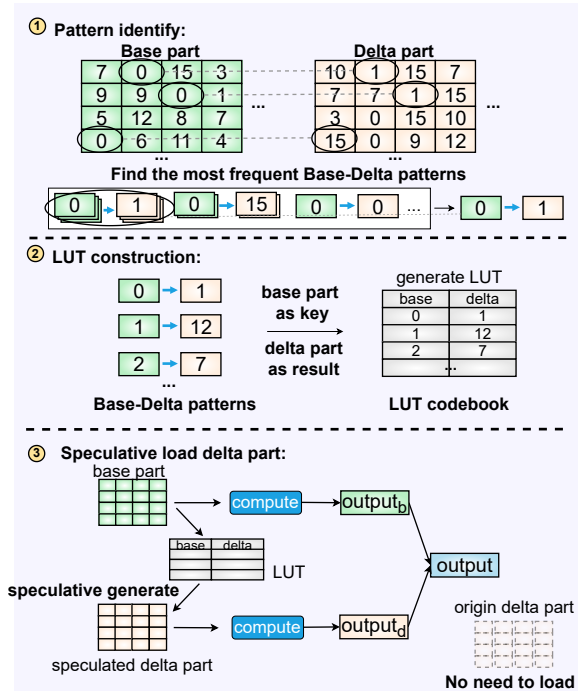


Figure 10. Pattern-based Speculative Reuse in EARTH. (a) Identification of frequent base–delta patterns. (b) LUT construction mapping base partitions to their associated deltas. (c) Runtime LUT queries allow speculative delta reuse, avoiding DRAM accesses for moderate-importance experts.

minimal bandwidth cost (Figure 8(a)). However, once base parts are handled efficiently, the *delta parts* emerge as the dominant bottleneck. To maintain accuracy (e.g., within 1%

of baseline perplexity), deltas must still be loaded for a large fraction of important experts; in our profiling, skipping deltas for more than 30% of them causes noticeable quality degradation. As a result, delta loading continues to contribute over 60% of the transfer overhead, limiting further efficiency gains.

This observation motivates a deeper question: how can we reduce delta-related costs in both loading and computation? Analyzing bit patterns in encoded experts, we uncover its significant redundancy. Figure 8(b) reveals that most patterns in delta parts are repeated, with specific $\langle \text{base}, \text{delta} \rangle$ pattern pairs dominating: Specially, Figure 8(c) shows that for base part $\langle 1111 \rangle$, 92.86% are between two possible patterns, and only 7.14% are other combinations. Notably, the number of dominant $\langle \text{base}, \text{delta} \rangle$ patterns is small, usually only tens of unique pairs across the evaluated model, and independent of model scale. Hence, even as the number of experts or parameters grows, the core set of recurring patterns remains unchanged. This high repetition rate across expert weights suggests that base computations can mirror delta computations.

5 Architecture

To address the bottleneck shifts identified in Section 4.4, we propose EARTH, a MoE inference accelerator designed to sustain high Processing Elements (PEs) utilization while mitigating expert-fetch stalls. In this section, we first provide an overview (Section 5.1), then detail the dataflow and memory layout (Section 5.3), and finally describe execution timeline optimizations that coordinate prefetching and delta decoding (Section 5.4).

5.1 Architecture Overview

We design **EARTH**, an accelerator that sustains high PE utilization under constrained memory bandwidth. Figure 9(a) shows the overall organization: a PE array coupled with high-bandwidth buffers and dispatchers, orchestrated by a main controller and gating module.

Compute Core Organization. The compute core executes vector–matrix multiplications in both the gating stage (Router–Linear) and feed-forward networks (Up/Down projection).

- **PE Array:** Each PE contains 16 Dot Product Units (DPUs) optimized for throughput on small vector tiles.
- **Weight Dispatcher:** Streams expert parameters from the weight buffer and broadcasts them across the PE array.
- **Activation Dispatcher:** Reorganizes activation tiles for spatial mapping to PEs, enabling balanced workload distribution.
- **Output Collector:** Accumulates partial sums from PEs and commits results to either off-chip memory or the next pipeline stage.

Memory Subsystem. The weight buffer stores experts in a bit-interleaved format, partitioned into a *base* part (W_{base}) and a *delta* part (W_{delta}). At runtime, only W_{base} and the LUT entries needed to reconstruct W_{delta} are fetched, deferring delta decoding until necessary. Both weight and token buffers are banked for fine-grained parallelism, with 4096 b/cycle and 1024 b/cycle interfaces to sustain PE demand.

Controller and Gating Module. The gating module integrates a top- k selector and arithmetic units for non-linearities and aggregation. The top- k unit identifies active experts and issues fetch requests; the arithmetic units (multipliers, adders, LUT-based non-linear units) implement softmax, GeLU, and high-precision accumulations. The main controller coordinates dispatch, gating, and accumulation to overlap memory transfers with computation.

EARTH incorporates MoE-specific hardware innovations to address dynamic data movement challenges. These components are purpose-built for MoE’s sparse and dynamic nature, distinguishing EARTH from general-purpose accelerators optimized for dense models or static sparsity.

5.2 Delta-aware Pattern Reuse

To address the delta loading bottleneck identified in Section 4.4, we integrate a pattern-based speculative reuse mechanism into EARTH’s architecture, leveraging the high redundancy across base–delta pairs. The key idea is to reuse deltas predicted from prefetched bases whenever possible, thus reducing delta traffic (Figure 10).

Pattern Identification and LUT Construction. We first analyze experts’ base and delta partitions to identify frequent

$\langle W_{\text{base}}, W_{\text{delta}} \rangle$ patterns. These are encoded into a LUT, where the base serves as the index and the corresponding delta as the entry. At runtime, the weight dispatcher can query the LUT with a fetched base to predict its delta, eliminating DRAM access for matched patterns.

Speculative Delta Loading. During inference, experts are categorized by their gating weights:

- **High-importance experts** (gating weight $>$ high threshold Δ): Always fetch full delta from DRAM to preserve accuracy.
- **Low-importance experts** (gating weight $<$ low threshold δ): Skip delta loading entirely, computing only with the base part, whose high-entropy content already provides adequate fidelity.
- **Moderate-importance experts** ($\delta \leq$ gating weight $\leq \Delta$): Apply *speculative delta reuse*. The base is used to query the LUT, and the predicted delta is reused without DRAM fetch.

Such a delta-aware reuse strategy eliminates redundant delta transfers for non-critical experts and overlaps delta prediction with PE computation, thereby shrinking the residual memory bottleneck and complementing EARTH’s prefetching and match-and-action scheduling.

5.3 Dataflow and Memory Layout

Figure 9(b) shows the dataflow. Weights are tiled into 4×16 blocks ($4 \text{ input} \times 16 \text{ output}$) to match PE granularity, while activations are tiled into 4-element vectors. The compute core adopts an *output-stationary* dataflow with asymmetric communication: weights are *unicasted* (since each expert is unique), while activations are *broadcast* (since they are shared across experts). This reduces traffic and balances utilization.

At runtime, execution proceeds in seven pipelined steps:

- 1 **On-chip load.** W_{base} and activations are fetched from DRAM into banked buffers.
- 2 **Weight dispatch.** The dispatcher reads W_{base} and reconstructs W_{delta} via LUT decoding as needed before unicasting tiles to target PEs.
- 3 **Activation dispatch.** Tiles are fetched from the token buffer and broadcast across PEs. LUT reconstruction amortizes expert-loading cost across tokens.
- 4 **PE compute.** Each PE performs parallel dot-products (16 outputs per tile) under output-stationary mapping, keeping partial sums local.
- 5 **Inter-PE accumulation.** The output collector aggregates results through an adder tree and accumulation buffer before forwarding them downstream.
- 6 **Non-linear and aggregation.** Depending on task type, the gating module applies: (a) top- k selection and softmax normalization for router outputs; (b) polynomial-approximated activations (e.g., GeLU); or (c) expert aggregation via weighted

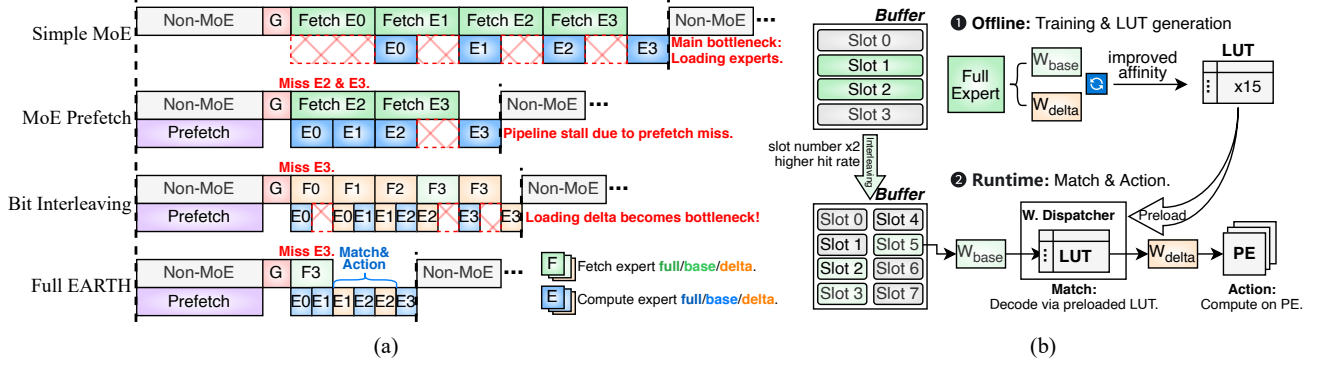


Figure 11. Execution timeline comparison and EARTH design. (a) *Timeline comparison*: End-to-end MoE inference latency under four methods—Simple MoE, MoE with prefetching, MoE with bit-interleaving, and the proposed EARTH. EARTH combines prefetching with a match-and-action schedule to eliminate the expert-fetch bottleneck and achieve near-full overlap of memory and computation. (b) *Bit-interleaving and match & action*: Each expert is split offline into base and delta partitions, trained for higher data affinity, and encoded into LUT entries. At runtime, only base partitions are prefetched on-chip, doubling effective buffer capacity and raising prefetch hit rate. Delta partitions are reconstructed on demand: the *match* phase decodes base parts via the LUT, and the *action* phase dispatches reconstructed weights to PEs.

accumulation. Arithmetic pipelines are unified by converting divisions into multiplications via Newton–Raphson iteration.

7 Write-back. Final outputs are written back to the token buffer and, if required, transferred to DRAM.

This organization reduces redundant weight transfers by partitioning experts into base and delta parts and using LUT-based reconstruction. It also maximizes compute utilization through output-stationary mapping and activation broadcasting. Pipelined overlap further hides memory latency. Together, these design choices effectively mitigate the expert-fetch bottleneck.

5.4 Timeline Optimization via Delta Reuse

Although speculative prefetching reduces stalls for base partitions, delta parts remain a critical bottleneck. To maintain high accuracy, important deltas must be fetched and decoded on time, and uncoordinated delta accesses can still stall PEs. This motivates a coordinated execution schedule that overlaps delta decoding with computation and maximizes the utilization of prefetched base partitions.

Figure 11 compares end-to-end inference timelines for four configurations: *Simple MoE*, *MoE with prefetching*, *MoE with bit-interleaving*, and *EARTH*. In the Simple MoE baseline, expert loading dominates execution time, leaving PEs idle. Prefetching reduces some stalls but is limited by buffer size and prefetch misses. Bit-interleaving improves prefetch hit rate by loading only base partitions on-chip, yet uncoordinated delta fetching still causes stalls.

EARTH addresses these limitations by combining bit-interleaving with a *match-and-action* execution schedule. By prefetching only compact base partitions, it doubles the number of resident experts in the on-chip buffer, significantly improving prefetch hit rate. The match-and-action mechanism

ensures that any required delta partition is decoded and dispatched just in time, preventing pipeline bubbles even under dynamic expert activation.

As illustrated in Figure 11(b), execution proceeds in two stages. Offline, each expert is split into a *base* partition (W_{base}) and a *delta* partition (W_{delta}), jointly trained to improve spatial affinity and encoded into compact LUT entries. At runtime, the *match* phase decodes W_{base} using the preloaded LUT, and the *action* phase dispatches reconstructed weights to the PE array. Weight decoding is overlapped with PE computation, ensuring that data arrives exactly when needed.

6 Evaluation

6.1 Experiment Setup

Platform and Configurations. We adopt a hybrid experimental methodology combining RTL synthesis, cycle-accurate simulation, and specialized memory modeling to ensure fidelity across physical, architectural, and system-level metrics:

- **RTL Implementation and Synthesis:** EARTH is fully implemented in Verilog at the RTL level, covering all critical components including the PE array, dual-entropy weight dispatcher, delta-reuse LUT, gating module, and accumulation logic. The design is synthesized with Synopsys Design Compiler using TSMC 28nm CMOS technology, with timing closure at 250 MHz and a nominal supply voltage of 1.25V. Synthesis reports are used to extract accurate area and static power consumption of digital logic (PEs, controllers, dispatchers).
- **On-Chip Memory Modeling:** On-chip storage structures (16 MB weight buffer, 1 MB token buffer) are

Table 1. Configuration of evaluated MoE models.

	Mixtral	Qwen	DeepSeek
#Layers	32	24	26
#Shared Experts	0	4	2
#Routed Experts	8	60	64
#Activated Experts	2	4	6
Shared Expert Size	-	(2048, 5632)	(2048, 1408)
Routed Expert Size	(4096, 14336)	(2048, 1408)	(2048, 1408)

modeled using CACTI 7 [3], a state-of-the-art memory modeling tool. CACTI is configured with TSMC 28nm process parameters to estimate dynamic energy, access latency, and area of SRAM banks, accounting for bank interleaving and port configuration (4096 b/cycle for weight buffer, 1024 b/cycle for token buffer).

- **Off-Chip Memory Configuration:** Off-chip memory is a HBM2E system with 8 stacks, 1024-bit interface per stack, and 256 GB/s theoretical peak bandwidth. HBM access energy (per bit) and latency (setup + data transfer) are derived from vendor specifications [31], with dynamic bandwidth contention modeled via trace-driven simulation.
- **Cycle-Accurate Simulation:** We develop a custom cycle-accurate simulator calibrated to RTL timing to evaluate end-to-end performance. The simulator models fine-grained behavior including PE compute pipelines, expert fetch/dispatch latency, delta decoding, LUT query, and HBM transaction scheduling. It captures key metrics: throughput, pipeline stalls, synchronization overhead, and compute-transfer overlap ratio. Traces are sampled from fixed-length inputs (512 tokens for CNN/DM, 2048 tokens for Longbench) to ensure consistency across models.

The accelerator’s core configuration is fixed for all experiments: 16 PEs, a radix-16 output collector, and banked buffers optimized for MoE’s sparse access patterns.

Models and Datasets. We evaluate EARTH on three representative MoE models: Mixtral-8x7B-Instruct (Mixtral) [20], DeepSeek-V2-Lite-Chat (DeepSeek) [26], and Qwen1.5-MoE-A2.7B (Qwen) [34]. Table 1 summarizes expert counts, sizes, and shared expert usage. Mixtral has fewer, larger experts without shared experts, while DeepSeek and Qwen have many smaller experts, including shared experts active for all tokens. This diversity enables evaluation of EARTH’s generality across MoE architectures. Datasets include CNN/DM [33] and Longbench (Gov_report) [2], with accuracy measured for QA tasks and Rouge-L for generation tasks.

Baselines. We compare EARTH with several representative MoE inference frameworks. **EdgeMoE** [41] is an FPGA-based accelerator for multi-task Vision Transformers, leveraging attention and patch reordering with minimal resource

Table 2. Evaluation results of EARTH on the Mixtral-8x7B-Instruct model. Datasets are evaluated using Rouge-L. **I.Exp.**, **M.Exp.** and **U.Exp.** means the percentage of important, moderate, and unimportant experts. **Load Red.** means the reduced loading requirements.

I. Exp.	M. Exp.	U. Exp.	CNN/DM	LongBench	Load Red.
Origin			35.6	34.3	13.1%
90%	0	10%	35.2	34.0	18.6%
80%	10%	10%	35.0	33.9	23.6%
80%	0	20%	34.1	31.9	23.6%
70%	10%	20%	32.6	29.3	29.2%
70%	0	30%	28.1	26.7	29.2%
20%	60%	20%	26.5	23.8	65.5%

Table 3. Evaluation results of EARTH on the Qwen1.5-MoE-A2.7B model. Datasets are evaluated using Rouge-L.

I. Exp.	M. Exp.	U. Exp.	CNN/DM	LongBench	Load Red.
Origin			31.2	26.4	8.8%
90%	0	10%	31.0	26.3	19.2%
80%	10%	10%	30.8	25.9	24.5%
80%	0	20%	29.7	25.2	24.5%
70%	10%	20%	28.5	24.6	30.3%
70%	0	30%	26.8	22.4	30.3%
30%	45%	25%	24.7	21.1	53.5%

Table 4. Evaluation results of EARTH on the DeepSeek-V2-Lite-Chat model. Datasets are evaluated using Rouge-L.

I. Exp.	M. Exp.	U. Exp.	CNN/DM	LongBench	Load Red.
Origin			28.7	25.5	18.3%
90%	0	10%	28.7	25.4	37.5%
80%	10%	10%	28.5	25.1	42.5%
80%	0	20%	28.1	24.3	42.5%
70%	10%	20%	27.4	23.8	48.1%
70%	0	30%	25.3	21.9	48.1%
30%	50%	20%	24.1	19.8	68.4%

usage. **AdapMoE** [44] is a GPU-centric scheduling framework that reduces on-demand loading overhead via adaptive expert prefetching and caching. **HybriMoE** [45] utilizes both CPU and GPU to improve hardware utilization through joint expert scheduling and cache hierarchy management. **DAOP** [42] dynamically allocates experts across CPU and GPU based on token-wise activation patterns. **APTMoE** [36] combines affinity-aware pipeline parallelism with hierarchical expert loading and demand-priority scheduling to enhance throughput and memory efficiency.

To assess performance under varying memory constraints, we define a key control metric, Cached Expert Ratio (CER), which denotes the ratio of available expert slots to the total number of experts in the model. And we evaluate the end-to-end latency to quantify inference performance, and we sample fixed-length traces from multiple datasets to ensure

consistency and comparability across different models and scheduling strategies.

6.2 Accuracy Results

We evaluate EARTH across Mixtral, Qwen, and DeepSeek under varying expert sensitivity configurations, with results summarized in Table 2, Table 3, and Table 4. EARTH achieves substantial reductions in expert loading while maintaining stable task accuracy. Retaining 80–90% of important experts incurs negligible loss, yet reduces overall expert loading by over 20%. Even under aggressive configurations with higher thresholds for unimportant experts and increased reliance on pattern-based delta reuse, EARTH maintains consistent performance with minimal degradation. Optimization potential varies with model characteristics. Mixtral, with larger but fewer experts, benefits from base-only prefetching, achieving load reduction without accuracy loss. DeepSeek exhibits stronger delta pattern repetition in long-sequence generation, enabling more aggressive speculative reuse and yielding up to 40% load reduction with negligible impact.

6.3 End-to-End Speedup

As shown in Figure 12, we evaluate EARTH across three MoE models (Mixtral, Qwen, DeepSeek), achieving consistent end-to-end speedups of 1.56×–2.10×. To further quantify EARTH’s efficiency in masking memory latency, we introduce the compute-transfer overlap ratio, which defined as $r_{Overlap} = 1 - T_{total}/T_{stall}$, where T_{total} is the actual end-to-end latency and T_{stall} is the theoretical latency if all expert-fetch stalls were unmasked. The ideal fully overlapped execution corresponds to 100% overlap, meaning no effective stall time, while practical overlap reflects how well EARTH coordinates prefetching, decoding, and computation.

Table 5 summarizes the ideal vs. practical overlap and speedup across models. For Mixtral, the theoretical ideal speedup (100% overlap) is 2.32×, and EARTH achieves 2.10× (90.5% of ideal) with 89% compute-transfer overlap. For Qwen, the ideal speedup is 2.21×, and EARTH attains 2.06× (93.2% of ideal) with 91% overlap. For DeepSeek, the ideal speedup is 1.83×, and EARTH reaches 1.72× (94.0% of ideal) with 86% overlap.

Gains stem from entropy-aware encoding, delta-aware speculative prefetching, and pattern reuse, which reduce memory traffic and computation while maintaining high prefetch accuracy. Performance varies by model: Mixtral benefits most due to large expert sizes (up to 2.10× over EdgeMoE), Qwen leverages high routed-expert counts for pattern reuse, and DeepSeek excels in long-sequence tasks through aggressive prefetching. Compared to baselines, EARTH avoids frequent stalls (EdgeMoE), reduces computation (AdapMoE / HybriMoE), and performs efficiently under limited cache (DAOP / APTMoE) via selective delta fetching and pattern

Table 5. Ideal vs. Practical Compute-Transfer Overlap and Speedup.

Model	Ideal Spd.	Practical Spd.	% of Ideal Spd.	Overlap
Mixtral	2.32×	2.10×	90.5%	89%
Qwen	2.21×	2.06×	93.2%	91%
DeepSeek	1.83×	1.72×	94.0%	86%

reuse. The remaining gap between practical and ideal overlap mainly stems from two factors: (1) delta-decoding latency for high-importance experts, which cannot be fully overlapped with computation due to dependency on base-part processing; and (2) synchronization overhead at expert-switch boundaries, where pipeline flushes are required for dynamic expert routing.

6.4 Energy and Area Breakdown

Figure 13 reports the detailed energy and area breakdown of EARTH. Energy is decomposed into DRAM, SRAM, PE array, output collector, and control logic; area is broken down into PE array, accumulators, LUTs, and control modules.

Energy. As shown in Figure 13(a), EARTH consumes 0.59× of the total energy compared to AdapMoE (1.71× lower) on the DeepSeek-V2-Lite-Chat workload, and reduces energy by 21.54% relative to EdgeMoE. **DRAM** energy drops to 0.57× (vs. 0.667× in EdgeMoE), due to (1) prefetching only the *base* part of each expert, which reduces data volume, and (2) higher prefetch hit rates combined with LUT-based handling of moderate experts, which eliminate most delta fetches. **SRAM** energy decreases to 0.68× (vs. 0.813× in EdgeMoE), as moderate and unimportant experts no longer load delta parts on-chip, significantly reducing memory traffic. **Compute logic**, including the PE array and output collector, consumes 0.60× energy.

Area. In terms of area, the total chip footprint is 27.52, mm². As shown in Figure 13(b), the PE array accounts for 77.55%, accumulators for 15.26%, and additional LUTs and control logic together for only 6.08%. Considering scalability, the LUT size remains constant regardless of model scale, this is because the dominant <base, delta> patterns are sparse and recurring, just have tens of entries and are independent on the number of experts or parameters. These results demonstrate that EARTH delivers substantial energy efficiency improvements with minimal area overhead.

6.5 Ablation Study

Effectiveness of EARTH Optimizations. We evaluate the impact of each optimization by enabling them progressively. As shown in Figure 14, vanilla MoE suffers severe expert-fetch stalls, limiting PE utilization and throughput. Naive prefetching mitigates part of the loading overhead, yielding a modest 1.12× speedup. Adding entropy-aware speculative prefetching further improves efficiency by preloading only

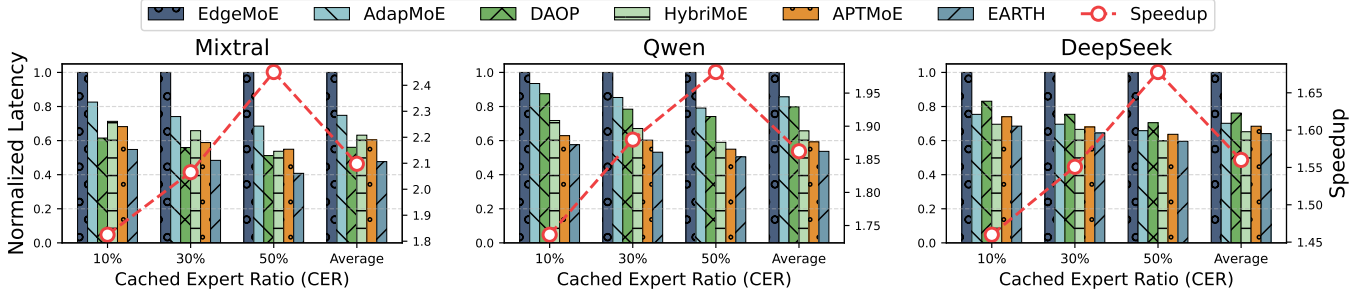


Figure 12. Inference performance comparison across different cache ratios.

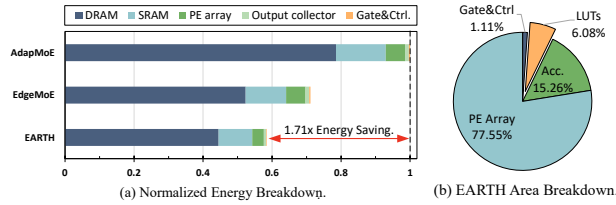


Figure 13. Energy and area breakdown of EARTH. (a) Normalized energy consumption of the MoE module in the DeepSeek-V2-Lite-Chat workload across AdapMoE, EdgeMoE, and EARTH. EARTH reduces energy by nearly 2 $\times$, mainly by optimizing dataflow and minimizing memory accesses. (b) Chip area composition of EARTH, showing the PE array dominates, while LUTs and control logic introduced by EARTH account for only 6.08%.

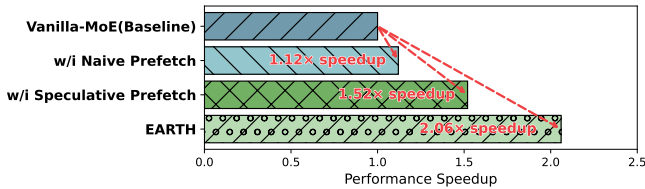


Figure 14. Impact of EARTH's optimization components.

high-information base partitions, effectively doubling on-chip expert capacity and reducing off-chip latency, achieving 1.52 $\times$ speedup. Finally, enabling full EARTH with delta reuse exploits common base-delta patterns to eliminate remaining delta-fetch overhead, reaching 2.06 $\times$ speedup.

Effectiveness of EARTH Data Format and Pattern Reuse. Figure 15 shows that EARTH's dual-entropy format and pattern reuse substantially improve both accuracy retention and load reduction. Compared to conventional INT truncation, which suffers sharp accuracy degradation once more than 10% of experts are approximated, EARTH maintains accuracy within 1% even when 20–25% of experts are approximated. Moreover, pattern reuse replaces redundant delta loads with lightweight table lookups, further amplifying bandwidth savings and reducing expert loading cost.

Memory Budget Sensitivity. As shown in Figure 16, naive prefetching yields low hit rates and limited gains.

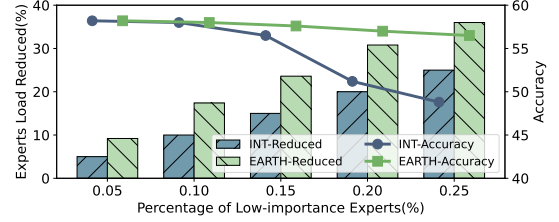


Figure 15. Load reduction and MMLU accuracy for INT and EARTH data format on DeepSeek-V2-Lite-Chat model.

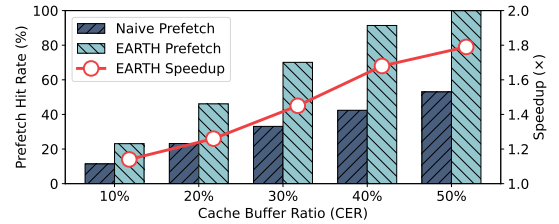


Figure 16. Impact of Cache Buffer Ratio.

EARTH achieves consistently higher hit rates, especially under tight budgets, by combining base-delta decomposition with FIFO-style refresh. As buffer size increases, the gap narrows since more experts can reside on-chip; at 50% capacity, nearly all routed experts are prefetched.

Layer-wise Pattern Reuse. Figure 17 shows EARTH's adaptive per-layer reuse ratios and speedup. Early layers use low reuse ($\approx 20 - 35\%$) to protect accuracy, while deeper layers allow higher reuse (up to 70%) without degradation. This allocation achieves an average 1.36 $\times$ speedup and maintains Rouge-L at 24.7 (vs. 21.3 under fixed reuse). By focusing pattern-reuse on tolerant experts and safeguarding sensitive ones, EARTH achieves a better accuracy-throughput tradeoff than uniform allocation.

7 Related Work

MoE Offloading Optimization. System-level techniques aim to reduce memory and communication overhead in MoE inference. Expert parallelism distributes experts across GPUs [21], mitigated by optimized patterns in DeepSpeedMoE [30] and FasterMoE [14]. Other approaches offload

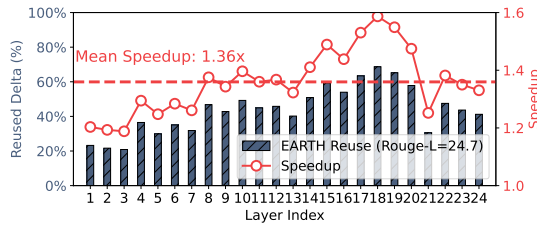


Figure 17. Layer-wise delta reuse ratios and speedup on Qwen for CNN/DM.

components to CPU or storage [9, 16, 25], or apply speculative loading with pre-gating [16], often requiring model changes or underutilizing CPU. In contrast, EARTH leverages a custom accelerator with entropy-aware encoding and delta-aware prefetching to reduce memory overhead efficiently without modifying model structure.

MoE Compression. Compression methods reduce model size and inference cost while preserving accuracy. Expert trimming [27] and Layer/Block Drop [15] remove experts or layers but may incur loss or communication overhead. SlimMoE [23] applies structured pruning, while quantization [11] and SVD-based decomposition [22] reduce parameter size with minimal retraining. Unlike these approaches, EARTH combines entropy-aware encoding with speculative prefetching on hardware, achieving high memory efficiency and speed without additional architectural changes.

MoE Accelerators. The irregular expert activation and intensive memory traffic of MoE models have motivated specialized hardware accelerators. Edge-MoE [32] reduces memory via task-level sparsity, UbiMoE [5] balances dense and sparse experts on FPGA, and M3ViT [24]/CoQMoE [6] explore model-accelerator co-design.

These works highlight the promise of specialized hardware for MoE acceleration, yet they primarily focus on optimizing computation patterns, or model-accelerator co-design. In contrast, EARTH uniquely integrates entropy-aware storage encoding with cross-token delta reuse, directly addressing the dominant bottlenecks of expert storage and runtime fetching.

8 Conclusion

We presented EARTH, a hardware-software co-design that improves the efficiency of MoE inference on resource-constrained platforms. EARTH integrates dual-entropy encoding, delta-aware speculative prefetching, and a specialized accelerator architecture. Together, these techniques reduce memory traffic by up to 48% and achieve up to 2.10 $\times$ speedup over state-of-the-art baselines, while maintaining near-baseline accuracy. EARTH makes large-scale MoE inference both efficient for resource-constrained deployment.

9 Acknowledgments

This work was partially supported by the National Key Research and Development Program of China (Grant No.2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), and Natural Science Foundation of Shanghai (Grant No.24ZR1433700). Haibing Guan is the corresponding author (E-mail: hbguan@sjtu.edu.cn).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Alvenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508* (2023).
- [3] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (June 2017), 25 pages. doi:10.1145/3085572
- [4] V. Chiley and Databricks NLP Team. 2024. DBRX: A new open-source large language model. <https://www.databricks.com/blog/2024/03/27/introducing-dbrx-new-state-art-open-llm>. Accessed: 2025-07-22.
- [5] Jiale Dong, Wenqi Lou, Zhendong Zheng, Yunqi Qin, Lei Gong, Chao Wang, and Xuehai Zhou. 2025. UbiMoE: A Ubiquitous Mixture-of-Experts Vision Transformer Accelerator With Hybrid Computation Pattern on FPGA. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. doi:10.1109/ISCAS56072.2025.11043956
- [6] Jiale Dong, Hao Wu, Zihao Wang, Wenqi Lou, Zhendong Zheng, Lei Gong, Chao Wang, and Xuehai Zhou. 2025. CoQMoE: Co-Designed Quantization and Computation Orchestration for Mixture-of-Experts Vision Transformer on FPGA. *arXiv preprint arXiv:2506.08496* (2025).
- [7] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. *Proceedings of Machine Learning and Systems* 6 (2024), 224–238.
- [8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv-2407.
- [9] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [11] Elias Frantar and Dan Alistarh. 2024. Qmoe: Sub-1-bit compression of trillion parameter models. *Proceedings of Machine Learning and Systems* 6 (2024), 439–451.
- [12] Hao Gu, Wei Li, Lujun Li, Qiyuan Zhu, Mark Lee, Shengjie Sun, Wei Xue, and Yike Guo. 2025. Delta decompression for moe-based llms compression. *arXiv preprint arXiv:2502.17298* (2025).
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [14] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. Fastmoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel*

- Programming. 120–134.
- [15] Shwai He, Daize Dong, Liang Ding, and Ang Li. 2024. Towards Efficient Mixture of Experts: A Holistic Study of Compression Techniques. *arXiv preprint arXiv:2406.02500* (2024).
 - [16] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S Lee, Anjali Sridhar, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. 2023. Towards moe deployment: Mitigating inefficiencies in mixture-of-expert (moe) inference. *arXiv preprint arXiv:2303.06182* (2023).
 - [17] Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. 2024. Mixture Compressor for Mixture-of-Experts LLMs Gains More. *arXiv preprint arXiv:2410.06270* (2024).
 - [18] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
 - [19] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991), 79–87.
 - [20] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
 - [21] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
 - [22] Wei Li, Lujun Li, Hao Gu, You-Liang Huang, Mark G Lee, Shengjie Sun, Wei Xue, and Yike Guo. [n. d.]. MoE-SVD: Structured Mixture-of-Experts LLMs Compression via Singular Value Decomposition. In *Forty-second International Conference on Machine Learning*.
 - [23] Zichong Li, Chen Liang, Zixuan Zhang, Ilgee Hong, Young Jin Kim, Weizhu Chen, and Tuo Zhao. 2025. SlimMoE: Structured Compression of Large MoE Models via Expert Slimming and Distillation. *arXiv preprint arXiv:2506.18349* (2025).
 - [24] Hanxue Liang, Zhiwen Fan, Rishov Sarkar, Ziyu Jiang, Tianlong Chen, Kai Zou, Yu Cheng, Cong Hao, and Zhangyang Wang. 2022. M3ViT: mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 2062, 17 pages.
 - [25] Junyang Lin, An Yang, Jinze Bai, Chang Zhou, Le Jiang, Xianyan Jia, Ang Wang, Jie Zhang, Yong Li, Wei Lin, et al. 2021. M6-10t: A sharing-delinking paradigm for efficient multi-trillion parameter pretraining. *arXiv preprint arXiv:2110.03888* (2021).
 - [26] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
 - [27] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. 2024. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. *arXiv preprint arXiv:2402.14800* (2024).
 - [28] Saeed Masoudnia and Reza Ebrahimpour. 2014. Mixture of experts: a literature survey. *Artificial Intelligence Review* 42, 2 (2014), 275–293.
 - [29] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
 - [30] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
 - [31] Rambus Inc. 2020. *White Paper: HBM2E and GDDR6: Memory Solutions for AI*. White Paper. Rambus Inc. <https://www.rambus.com/hbm2e-and-gddr6-memory-solutions-for-ai/>
 - [32] Rishov Sarkar, Hanxue Liang, Zhiwen Fan, Zhangyang Wang, and Cong Hao. 2023. Edge-MoE: Memory-Efficient Multi-Task Vision Transformer Architecture with Task-Level Sparsity via Mixture-of-Experts. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 01–09. doi:10.1109/ICCAD57390.2023.10323651
 - [33] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368* (2017).
 - [34] Qwen Team. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* (2024).
 - [35] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [36] Yuanxin Wei, Jiangsu Du, Jiazhi Jiang, Xiao Shi, Xianwei Zhang, Dan Huang, Nong Xiao, and Yutong Lu. 2024. APTMoE: Affinity-Aware Pipeline Tuning for MoE Models on Bandwidth-Constrained GPU Nodes. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
 - [37] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100* (2022).
 - [38] xAI. 2024. Grok-1: Open-source release of 314B parameter mixture-of-experts model. <https://x.ai/blog/grok-1-open-source>. Accessed: 2025-07-22.
 - [39] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Offloading-efficient moe model serving. *arXiv preprint arXiv:2401.14361* (2024).
 - [40] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. 2024. MoE-I²: Compressing Mixture of Experts Models through Inter-Expert Pruning and Intra-Expert Low-Rank Decomposition. *arXiv preprint arXiv:2411.01016* (2024).
 - [41] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. Edgemoe: Fast on-device inference of moe-based large language models. *arXiv preprint arXiv:2308.14352* (2023).
 - [42] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. 2025. DAOP: Data-Aware Offloading and Predictive Pre-Calculation for Efficient MoE Inference. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.
 - [43] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
 - [44] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
 - [45] Shuzhang Zhong, Yanfan Sun, Ling Liang, Runsheng Wang, Ru Huang, and Meng Li. 2025. HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference. *arXiv preprint arXiv:2504.05897* (2025).